AI Pipelines — API Reference

May 2026

AI Pipelines — API Reference

Complete reference for all `ai.*` SQL functions defined in `ai_pipeline_functions.sql`.

Overview

The AI pipeline layer is a declarative SQL API built on top of pg_durable. You describe **what** you want (source, steps, sink) and the system builds a crash-safe durable execution graph that runs inside PostgreSQL.

Dependencies: `pg_durable`, `vector` (pgvector), `azure_ai`

```
Source → Step 1 → Step 2 → ... → Sink
         (chunk)   (embed)        (table)
```

Tables

`ai.pipelines`

Stores registered pipeline definitions.

Column	Type	Description
`name`	TEXT (PK)	Unique pipeline identifier
`source_config`	JSONB	Source descriptor (from `ai.table_source` / `ai.file_source`)
`steps`	JSONB[]	Array of step descriptors
`sink_config`	JSONB	Sink descriptor (from `ai.table_sink` or auto-created)
`trigger_type`	TEXT	`manual`, `on_change`, or `schedule`
`options`	JSONB	Additional pipeline options
`created_by`	TEXT	Role that created the pipeline
`created_at`	TIMESTAMPTZ	Creation timestamp
`paused`	BOOLEAN	Whether the pipeline is paused

`ai.pipeline_runs`

Tracks each execution of a pipeline.

Column	Type	Description
`id`	`BIGSERIAL` (PK)	Run identifier
`pipeline_name`	`TEXT` (FK)	Pipeline this run belongs to
`instance_id`	`TEXT`	`df.start()` instance ID
`status`	`TEXT`	`pending`, `running`, `completed`, `failed`
`rows_processed`	`INT`	Count of rows processed
`error`	`TEXT`	Error message if failed
`started_at`	`TIMESTAMPTZ`	Run start time
`completed_at`	`TIMESTAMPTZ`	Run completion time

`ai.pipeline_checkpoints`

Tracks incremental processing position per pipeline.

Column	Type	Description
`pipeline_name`	`TEXT` (PK, FK)	Pipeline name
`last_value`	`TEXT`	Last processed incremental column value
`last_run_at`	`TIMESTAMPTZ`	When last checkpoint was saved
`total_processed`	`BIGINT`	Cumulative rows processed

`ai.cost_log`

Logs token usage and estimated cost per step.

Column	Type	Description
`id`	`BIGSERIAL` (PK)	Log entry ID
`pipeline_name`	`TEXT` (FK)	Pipeline name
`run_id`	`BIGINT` (FK)	Pipeline run ID
`step_name`	`TEXT`	Step that incurred the cost
`model`	`TEXT`	Model used
`input_tokens`	`INT`	Input tokens consumed
`output_tokens`	`INT`	Output tokens generated
`estimated_cost`	`NUMERIC(12,6)`	Estimated dollar cost

Source Constructors

`ai.table_source()`

Define a database table as the pipeline's input.

```
ai.table_source(
    table_name          TEXT,
    incremental_column  TEXT DEFAULT NULL,
    schema_name         TEXT DEFAULT 'public',
    filter              TEXT DEFAULT NULL
) → JSONB
```

Parameter	Description
`table_name`	Name of the source table
`incremental_column`	Column used for change tracking (e.g. `updated_at`). Only new/changed rows are processed on subsequent runs.
`schema_name`	Schema of the source table
`filter`	Optional SQL `WHERE` clause fragment

Example:

```
ai.table_source('documents', incremental_column => 'updated_at')
```

`ai.file_source()`

Define a file/blob source (not yet fully implemented).

```
ai.file_source(
    uri     TEXT,
    formats TEXT[] DEFAULT ARRAY['pdf', 'txt', 'md']
) → JSONB
```

Sink Constructors

`ai.table_sink()`

Define where processed results are written. If omitted in `ai.create_pipeline()`, a sink table is auto-created as `public.<pipeline_name>_output`.

```
ai.table_sink(
    table_name          TEXT,
    schema_name         TEXT DEFAULT 'public',
    columns             TEXT[] DEFAULT NULL,
    on_conflict         TEXT[] DEFAULT NULL,
    on_conflict_action  TEXT DEFAULT 'update'
) → JSONB
```

Parameter	Description
`table_name`	Destination table name
`schema_name`	Destination schema
`on_conflict`	Columns for `ON CONFLICT` clause
`on_conflict_action`	`'update'` or `'nothing'`

Example:

```
ai.table_sink('product_vectors', on_conflict => ARRAY['product_id'], on_conflict_a
```

Step Constructors

Each step returns a `JSONB` descriptor. Steps are composed into an array and passed to `ai.create_pipeline()`.

`ai.chunk()`

Split text into overlapping segments for downstream embedding or analysis.

```
ai.chunk(
    input_column TEXT,
    method       TEXT DEFAULT 'recursive',
    chunk_size   INT DEFAULT 512,
    overlap      INT DEFAULT 64
) → JSONB
```

Parameter	Description
`input_column`	Column containing the text to chunk
`method`	Chunking strategy (`'recursive'`)
`chunk_size`	Target characters per chunk
`overlap`	Character overlap between consecutive chunks

Output columns added: `doc_id`, `chunk_index`, `chunk_text`

`ai.embed()`

Generate vector embeddings via Azure OpenAI.

```
ai.embed(
    model        TEXT,
    input_column TEXT,
    batch_size   INT DEFAULT 100,
    dimensions   INT DEFAULT NULL
) → JSONB
```

Parameter	Description
`model`	Azure OpenAI deployment name (e.g. `'text-embedding-3-small'`)
`input_column`	Column to embed
`batch_size`	Rows per API call
`dimensions`	Vector dimensions (e.g. 1536). `NULL` = model default.

Output column added: `embedding` (`vector`)

ai.extract()

Extract structured fields from text via LLM.

```
ai.extract(
    model        TEXT,
    input_column TEXT,
    data         TEXT[] DEFAULT NULL,
    fields       JSONB DEFAULT NULL
) → JSONB
```

Parameter	Description
`model`	LLM deployment name (e.g. `'gpt-5-mini'`)
`input_column`	Column containing text to analyze
`data`	Array of field descriptions (e.g. `ARRAY['category - product category', 'audience - target user']`)
`fields`	Alternative: JSONB field spec

Output column added: `extracted` (`JSONB`)

ai.generate()

Generate text via LLM using a prompt template.

```
ai.generate(
    model           TEXT,
    prompt_template TEXT,
    input_column    TEXT DEFAULT NULL,
    max_tokens      INT DEFAULT 1024
) → JSONB
```

Parameter	Description
`model`	LLM deployment name

Parameter	Description
`prompt_template`	Prompt with `{column_name}` placeholders
`input_column`	Column for template substitution
`max_tokens`	Maximum response tokens

Output column added: `generated` (TEXT)

ai.rank()

Re-rank results by relevance using an LLM.

```
ai.rank(
    model        TEXT,
    query_column TEXT,
    doc_column   TEXT,
    top_k        INT DEFAULT 10
) → JSONB
```

Parameter	Description
`model`	Ranking model deployment
`query_column`	Column with the search query
`doc_column`	Column with the document text
`top_k`	Number of top results to keep

Output column added: `rank_score` (NUMERIC)

ai.request_approval()

Pause the pipeline and wait for a human signal before continuing.

```
ai.request_approval(
    content TEXT,
    notify  TEXT DEFAULT NULL,
    timeout INT DEFAULT 3600
) → JSONB
```

Parameter	Description
`content`	Column whose value is presented for review
`notify`	Optional notification channel
`timeout`	Seconds to wait before timing out (default 1 hour)

Approval signal name: `pipeline_<pipeline_name>_approval`

To approve a paused pipeline:

```
SELECT df.signal('<instance_id>', 'pipeline_<pipeline_name>_approval');
```

ai.parse_document()

Parse documents from various formats (PDF, TXT, MD).

```
ai.parse_document(
    source  TEXT,
    format  TEXT DEFAULT 'auto',
    options JSONB DEFAULT '{}'
) → JSONB
```

Pipeline Lifecycle Functions

ai.create_pipeline()

Register a new pipeline definition.

```
ai.create_pipeline(
    name    TEXT,
    source  JSONB,
    steps   JSONB[],
    sink    JSONB DEFAULT NULL,
    trigger TEXT DEFAULT 'manual',
    options JSONB DEFAULT '{}'
) → TEXT
```

Parameter	Description
name	Unique pipeline name (alphanumeric, _, -)
source	Source descriptor (from ai.table_source() or ai.file_source())
steps	Array of step descriptors
sink	Sink descriptor. If NULL, auto-creates public.<name>_output
trigger	'manual', 'on_change', or 'schedule'
options	Additional options as JSONB

Returns: Success message.

Example:

```
SELECT ai.create_pipeline(
    name   => 'rag_pipeline',
    source => ai.table_source('documents', incremental_column => 'updated_at'),
    steps  => ARRAY[
        ai.chunk(input_column => 'content'),
```

```
        ai.embed(model => 'text-embedding-3-small', input_column => 'chunk_text',
                 dimensions => 1536)
    ],
    trigger => 'on_change'
);
```

ai.run()

Execute a pipeline. Builds a durable execution graph and starts it via `df.start()`.

```
ai.run(pipeline_name TEXT) → TEXT
```

Returns: Instance ID (8-char hex string). Use with `df.status()`, `df.result()`, dashboard, etc.

ai.drop()

Remove a pipeline and its change trigger (if any).

```
ai.drop(pipeline_name TEXT) → TEXT
```

ai.pause()

Pause a pipeline. Change triggers still fire but runs are skipped.

```
ai.pause(pipeline_name TEXT) → TEXT
```

ai.resume()

Resume a paused pipeline.

```
ai.resume(pipeline_name TEXT) → TEXT
```

ai.backfill()

Reset the checkpoint and reprocess all source data from scratch.

```
ai.backfill(
    pipeline_name TEXT,
    batch_size    INT DEFAULT NULL
) → TEXT
```

Returns: Instance ID for the backfill run.

Monitoring Functions

ai.status()

Get pipeline status with latest run info.

```
ai.status(pipeline_name TEXT DEFAULT NULL)
→ TABLE(name, trigger_type, paused, last_run_status, last_run_at,
        total_runs, total_processed, last_instance, df_status)
```

Pass NULL (or omit) to see all pipelines.

ai.list_pipelines()

List all registered pipelines.

```
ai.list_pipelines()
→ TABLE(name, source_type, step_count, trigger_type, paused, created_at, created_by
```

ai.explain()

Show a human-readable execution plan for a pipeline.

```
ai.explain(pipeline_name TEXT) → TEXT
```

Example output:

```
Pipeline: rag_pipeline
Trigger:  on_change
──────────────────────────────────
  [SOURCE] public.documents (incremental: updated_at)
     │
     ▼
  [STEP 1] CHUNK (column=content, method=recursive, size=512, overlap=64)
     │
     ▼
  [STEP 2] EMBED (model=text-embedding-3-small, column=chunk_text, batch=100)
     │
     ▼
  [SINK] public.rag_pipeline_output
```

ai.result()

Get the result of a pipeline run.

```
ai.result(
    pipeline_name TEXT,
    run_number    INT DEFAULT NULL  -- NULL = latest
)
→ TABLE(run_id, instance_id, status, started_at, completed_at,
        rows_processed, error, df_result)
```

ai.cost_summary()

View aggregated token usage and estimated costs.

```
ai.cost_summary(pipeline_name TEXT DEFAULT NULL)
→ TABLE(name, step_name, model, total_input, total_output, total_cost, call_count)
```

ai.wait_for_completion()

Block until a pipeline run finishes (or times out).

```
ai.wait_for_completion(
    pipeline_name TEXT,
    timeout_secs  INT DEFAULT 300
) → TEXT
```

Returns: Final status string (`'completed'`, `'failed'`, etc.)

Trigger Behavior

When `trigger => 'on_change'` is set:

1. An `AFTER INSERT OR UPDATE` statement-level trigger is created on the source table.
2. On any write, the trigger calls `ai.run()` — but only if no run is currently in progress (debounced).
3. Incremental processing uses `incremental_column` to skip already-processed rows.
4. Pausing a pipeline keeps the trigger active but skips execution.

End-to-End Examples

Minimal RAG Pipeline

```
-- 1. Create source table
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 2. Create pipeline (auto-creates documents_output sink)
```

```
SELECT ai.create_pipeline(
    name   => 'rag_pipeline',
    source => ai.table_source('documents', incremental_column => 'updated_at'),
    steps  => ARRAY[
        ai.chunk(input_column => 'content'),
        ai.embed(model => 'text-embedding-3-small', input_column => 'chunk_text',
                 dimensions => 1536)
    ],
    trigger => 'on_change'
);

-- 3. Run it
SELECT ai.run('rag_pipeline');

-- 4. Check status
SELECT * FROM ai.status('rag_pipeline');

-- 5. Query results
SELECT doc_id, chunk_text, embedding
FROM rag_pipeline_output
LIMIT 5;
```

Pipeline with Extraction + Approval + Generation

```
SELECT ai.create_pipeline(
    name   => 'enriched_pipeline',
    source => ai.table_source('documents', incremental_column => 'updated_at'),
    steps  => ARRAY[
        ai.chunk(input_column => 'content'),
        ai.extract(
            model        => 'gpt-5-mini',
            input_column => 'chunk_text',
            data         => ARRAY[
                'category - product category',
                'audience - target audience',
                'price_tier - budget, mid, or premium'
            ]
        ),
        ai.request_approval(content => 'chunk_text', timeout => 3600),
        ai.embed(model => 'text-embedding-3-small', input_column => 'chunk_text',
                 dimensions => 1536),
        ai.generate(
            model           => 'gpt-5-mini',
            input_column    => 'chunk_text',
            prompt_template => 'Write a one-sentence summary of: {chunk_text}'
        )
```

```
    ],
    trigger => 'on_change'
);
```

Upsert Pipeline with Conflict Handling

```
SELECT ai.create_pipeline(
    name   => 'product_enrichment',
    source => ai.table_source('products', incremental_column => 'updated_at'),
    steps  => ARRAY[
        ai.embed(model => 'text-embedding-3-small', input_column => 'description')
        ai.extract(model => 'gpt-4o', input_column => 'description',
                   data => ARRAY['category', 'brand', 'key_features'])
    ],
    sink    => ai.table_sink('product_vectors',
                  on_conflict => ARRAY['product_id'],
                  on_conflict_action => 'update'),
    trigger => 'on_change'
);
```

Files in This Folder

File	Purpose
ai_pipeline_functions.sql	All function definitions — load this first
demo_rag_pipeline_short.sql	Short demo script (~2 min)
demo_rag_pipeline.sql	Full demo with vector search
demo_rag_setup.sql	Demo setup with sample data
rag_pipeline.sql	Extended RAG example with backfill
feature_enrichment.sql	Product enrichment pipeline
human_approval.sql	Support triage with approval gate